Diseño probado de una solución orientada a objetos

Actividad 06

Sistema de gestión de envíos nacionales e internacionales

Escuela Superior de Empresa Ingeniería y Tecnología ESEIT

Programación Orientada a Objetos II

Jeison Martínez Ramírez

Valerie Pamela Suárez Corredor

Repositorio del proyecto: https://github.com/JEIJEIMARTINEZ/ACT06-Logistica

Síntesis del caso

LogiTrans es una empresa que necesita registrar envíos nacionales e internacionales, calcular sus costos y consultar su avance. Cuando las tarifas se calculan en lugares diferentes, pueden aparecer inconsistencias; además, una guía repetida o un cambio de estado incorrecto dificulta el seguimiento. La solución propuesta concentra las reglas verificables en un modelo orientado a objetos y conserva evidencia de su funcionamiento mediante pruebas automatizadas.

El alcance incluye la creación de envíos con datos válidos, la asociación con una ruta y una tarifa, el cálculo monetario en pesos colombianos, el registro de guías únicas y la secuencia de entrega. Los dos tipos de envío comparten un contrato de comportamiento y se procesan a través del supertipo Envio. Cada envío administra sus propios eventos de seguimiento; las rutas y las tarifas pueden compartirse porque son inmutables.

El proyecto se ejecuta en consola con Java y Maven. JUnit 5 verifica escenarios válidos, errores, límites y sustitución polimórfica. La matriz de trazabilidad relaciona reglas, UML, código y pruebas. Dos refactorizaciones se comparan contra una línea base para conservar resultados. Quedan fuera del alcance la autenticación, la base de datos, las tarifas comerciales reales y la integración con transportadoras.

Plan técnico

Campo	Desarrollo
Problema y alcance	LogiTrans necesita registrar, cotizar y seguir envíos nacionales e internacionales. Incluye validación, cálculo, guías únicas y estados. Excluye pagos, autenticación, persistencia e integración con transportadoras.
Reglas de negocio	R1 a R8. Se verifican mediante resultados monetarios, aceptación o rechazo de datos, transiciones de estado, unicidad y protección de colecciones.
Clases y responsabilidades	Envio: datos comunes y seguimiento; EnvioNacional: costo nacional; EnvioInternacional: costo con flete y arancel; Ruta: origen y destino; Tarifa: componente común del costo; RegistroEnvios: guías y total; EventoSeguimiento: estado alcanzado.
Relaciones UML	Herencia de Envio en dos subtipos. Asociaciones de Envio con Ruta y Tarifa y de RegistroEnvios con Envio. Composición de Envio con EventoSeguimiento.
Contrato de la jerarquía	calcularCosto devuelve un importe positivo en COP con dos decimales, de forma determinista y sin alterar estado o historial. Ambos subtipos admiten la misma secuencia de estados.
Escenarios de prueba	Éxito, nulos, duplicados, pesos fuera y dentro de los límites, precisión monetaria, transiciones válidas e inválidas y contrato aplicado a ambos subtipos.
Trazabilidad	ID R1 a R8 vinculados con elemento UML, clase o método Java, prueba JUnit y resultado de ejecución.
Plan de refactorización	Extraer a Tarifa la fórmula duplicada; separar la validación de la inserción en RegistroEnvios. Conservar la misma suite y comparar resultados antes y después.
Distribución del equipo	Distribución propuesta: Jeison, implementación, ejecución, reglas y UML; Valerie, pruebas y matriz. Revisión cruzada antes de integrar. Ramas previstas: feature/reglas-uml, feature/implementación y feature/pruebas.

Requerimientos y reglas de negocio

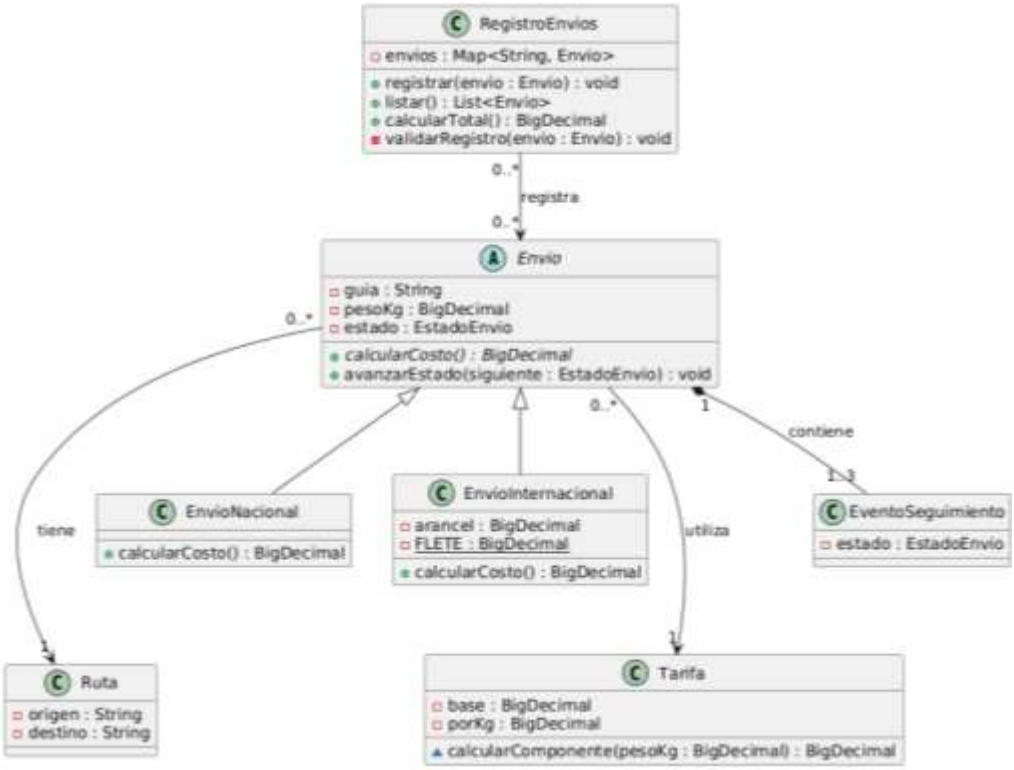
ID	Regla y criterio de aceptación
R1	Guía no vacía, normalizada con strip; peso en (0, 50] kg; ruta y tarifa obligatorias. Ambos subtipos rechazan datos inválidos con IllegalArgumentException.
R2	Base y precio por kg positivos con hasta dos decimales. Nacional = base + peso × precio/kg. Internacional añade 45000 COP y arancel. Redondeo HALF_UP a dos decimales al final.
R3	Origen y destino no vacíos y distintos al ignorar mayúsculas y espacios exteriores. Ruta es inmutable y compartible. Datos inválidos producen IllegalArgumentException.
R4	Arancel internacional obligatorio, no negativo y con máximo dos decimales. Cero es válido; null, negativos o precisión excesiva producen IllegalArgumentException.
R5	Cada registro admite una guía normalizada una sola vez. La comparación distingue mayúsculas. Envío null o duplicado produce IllegalArgumentException sin modificar el registro.
R6	Secuencia única CREADO → EN_TRANSITO → ENTREGADO. Saltos, retrocesos, repeticiones y null producen IllegalStateException sin alterar estado ni historial.
R7	Cada envío crea y posee de uno a tres eventos inmutables. El exterior no puede insertar eventos ni modificar estructuralmente las listas públicas: se devuelven copias no modificables.
R8	Ambos subtipos satisfacen el contrato de Envio. RegistroEnvios calcula el total mediante el supertipo. equals y hashCode dependen solo de la guía estable; toString muestra guía y estado.

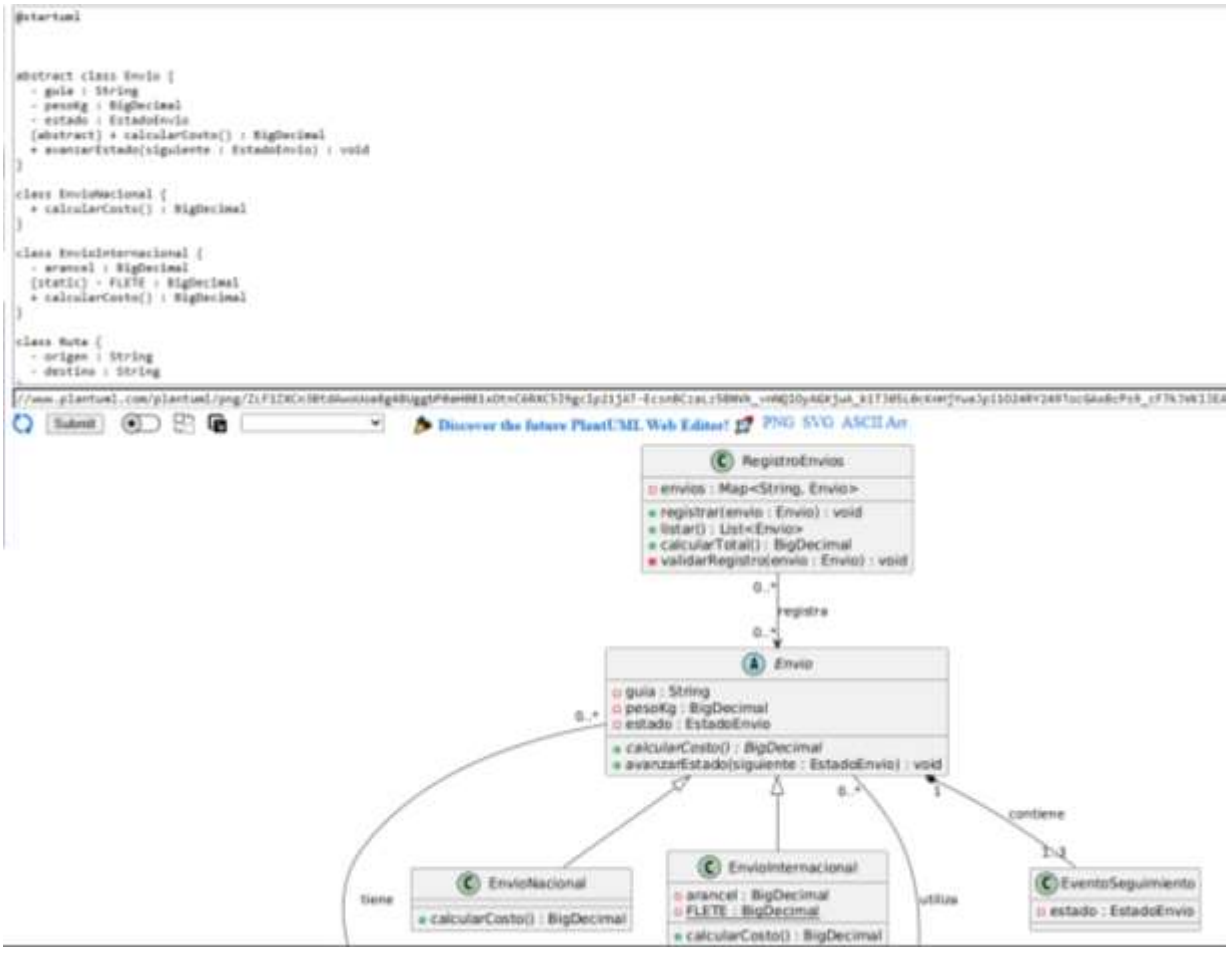
Supuestos del caso: importes académicos en COP, sin impuestos adicionales ni conversión monetaria. El peso no tiene un mínimo positivo fijo; se admite cualquier BigDecimal mayor que cero hasta 50 kg.

Ejemplo verificable: base 10000 COP, precio 2000 COP/kg y peso 2 kg producen 14000.00 COP para el nacional. Con arancel 5000 COP, el internacional cuesta 64000.00 COP. El total mixto es 78000.00 COP; un registro vacío totaliza 0.00 COP.

Modelo UML

Diagrama simplificado con las siete clases del dominio y sus relaciones. Se muestran los atributos y métodos principales; los constructores, getters y métodos auxiliares se conservan en el código Java. EstadoEnvio representa CREADO, EN_TRANSITO y ENTREGADO.





La generalización permite procesar EnvioNacional y EnvioInternacional como Envio. Cada envío tiene una Ruta y una Tarifa; ambas pueden compartirse entre cero o muchos envíos porque son inmutables. RegistroEnvios se asocia con cero o muchos envíos; un envío puede existir sin registro o pertenecer a varios registros.

La composición vincula cada envío con uno a tres eventos creados internamente: uno inicial y uno por transición válida. No existe una operación pública para transferir o insertar eventos externos. La visibilidad de paquete del constructor de EventoSeguimiento protege la API frente a clientes de otros paquetes; el código interno debe respetar esta propiedad.

Visibilidades: + público; - privado; # protegido; ~ paquete. Los tipos completos de los parámetros de construcción están definidos en Java. Main ejecuta la demostración y no forma parte del modelo de dominio.

Implementación Java

Paquete: co.edu.eseit.logistica. Los atributos son privados. La guía, el peso, la ruta, la tarifa y el arancel se fijan en la construcción. Los constructores validan R1 a R4. BigDecimal representa el dinero y el redondeo se realiza al final del cálculo.

Sobrescritura y cálculo polimórfico

```
// Tarifa: fórmula común después de la primera refactorización.
BigDecimal calcularComponente(BigDecimal pesoKg) {
    return base.add(pesoKg.multiply(porKg));
}
// EnvioNacional
@Override public BigDecimal calcularCosto() {
    return getTarifa().calcularComponente(getPesoKg())
            .setScale(2, RoundingMode.HALF_UP);
}
// EnvioInternacional
@Override public BigDecimal calcularCosto() {
    return getTarifa().calcularComponente(getPesoKg())
            .add(FLETE).add(arancel).setScale(2, RoundingMode.HALF_UP);
}
// RegistroEnvios: cliente del supertipo.
public BigDecimal calcularTotal() {
    return envios.values().stream().map(Envio::calcularCosto)
            .reduce(new BigDecimal("0.00"), BigDecimal::add);
}
```

Encapsulamiento y estados

```
public final List<EventoSeguimiento> getHistorial() {
    return List.copyOf(historial);
}
public final void avanzarEstado(EstadoEnvio siguiente) {
    boolean valido = (estado == EstadoEnvio.CREADO
            && siguiente == EstadoEnvio.EN_TRANSITO)
        || (estado == EstadoEnvio.EN_TRANSITO
            && siguiente == EstadoEnvio.ENTREGADO);
    if (!valido) throw new IllegalStateException("Transición no permitida");
    estado = siguiente;
    historial.add(new EventoSeguimiento(estado));
}
```

Envio declara calcularCosto como abstracto y ambos subtipos lo sobrescriben con @Override. avanzarEstado es final para conservar el mismo contrato de seguimiento. equals y hashCode también son finales y usan únicamente la guía normalizada. Así, la igualdad permanece estable aunque cambie el estado.

Pruebas unitarias y evidencias

La suite LogisticaTest utiliza JUnit Jupiter 5.11.4. Se ejecutan 31 casos, incluidas las combinaciones parametrizadas. Las pruebas verifican resultados y excepciones concretos; no dependen de servicios externos ni de entradas aleatorias.

Reglas	Casos verificados
R1	Pesos −1, 0 y 50.001: rechazo en ambos tipos. Pesos 0.001 y 50: aceptación. Guías null, vacías o espacios y referencias obligatorias null: rechazo.
R2 y R4	Tarifas positivas; rechazo de cero, negativos, null y precisión excesiva. Redondeo 0.015 a 0.02. Arancel cero válido; negativo, null o más de dos decimales inválidos.
R3 y R5	Ruta normalizada y compartible; rechazo de extremos vacíos o equivalentes. Dos guías distintas válidas; duplicados y envío null rechazados sin alterar el registro.
R6 y R7	Secuencia completa y rechazo de saltos, retrocesos, repeticiones y null. Copias no modificables e independientes; cada envío crea sus propios eventos.
R8	Contrato parametrizado sobre ambos subtipos: costo esperado, repetición sin efectos y secuencia común. Total mixto y vacío. Igualdad y hash estables al avanzar estado.

Prueba del contrato y supertipo

contratoDelSupertipoConservaCostoEstadoEHistorial recibe referencias Envio de ambos tipos. Comprueba 14000.00 COP para el nacional y 64000.00 COP para el internacional, costo positivo y determinista, estado e historial sin cambios por el cálculo, avance hasta ENTREGADO y rechazo de una transición posterior.

Resultados de ejecución

Etapa	Casos	Fallos	Errores	Omitidos
Línea base	31	0	0	0
Después de refactorización 1	31	0	0	0
Después de refactorización 2	31	0	0	0
Revisión limpia final	31	0	0	0

Entorno: Amazon Corretto OpenJDK 21.0.12.1, Maven 3.9.9, JUnit 5.11.4 y Linux x86_64. Comando: mvn -B -ntp clean test. Reportes: evidencias/01_linea_base.txt, 02_refactor_tarifa.txt, 03_refactor_registro.txt y 05_revision_limpia.txt, acompañados de XML JUnit.

```
Tests run: 31, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Se alteró temporalmente el flete de 45000 a 45001 COP: se detectaron siete fallos y BUILD FAILURE. El valor correcto se restauró antes de la revisión final. El reporte 04_mutacion_detectada.txt y mutacion_flete.diff conservan esa evidencia. La demostración final arroja 14000.00, 64000.00 y total 78000.00 COP.

Matriz de trazabilidad

Cada ID se vincula con el modelo, el código y una prueba identificable. Los nombres corresponden a métodos de LogisticaTest. El resultado se refiere a la revisión limpia final.

ID	Elemento UML	Clase o método Java	Prueba JUnit	Resultado
R1	Envio y subtipos	Envio constructor	rechazaPesoFueraDelIntervaloEnAmbosSubtipos; aceptaPesosValidosCercanosALosLimites; rechazaGuiaAusenteEnAmbosSubtipos; rechazaReferenciasObligatoriasNulasEnAmbosSubtipos	Aprobado
R2	Tarifa y subtipos	Tarifa constructor y calcularComponente; calcularCosto	rechazaTarifasNoPositivasOPrecisionExcesiva; rechazaTarifasNulas; aceptaTarifaMinimaYRedondeaUnaSolaVez; contratoDelSupertipoConservaCostoEstadoEHistorial	Aprobado
R3	Asociación Envio a Ruta	Ruta constructor	rutaNormalizaEspaciosYPermiteCompartirReferencias; rechazaExtremosDeRutaAusentes; rechazaRutaConOrigenYDestinoEquivalentes	Aprobado
R4	EnvioInternacional	Constructor y calcularCosto	rechazaArancelNegativoOPrecisionExcesiva; aceptaArancelCeroYRechazaNulo	Aprobado
R5	RegistroEnvios a Envio	registrar y validarRegistro	registraEnviosDistintosYRechazaGuiaDuplicadaSinAlterarRegistro; rechazaRegistroNuloYConservaRegistroVacio	Aprobado
R6	Envio y EstadoEnvio	avanzarEstado	rechazaSaltosRepeticionesRetrocesosYNulosSinAlterarHistorial; contratoDelSupertipoConservaCostoEstadoEHistorial	Aprobado
R7	Composición Envio a EventoSeguimiento	getHistorial; avanzarEstado; listar	protegeColeccionesYPropiedadDelHistorial	Aprobado
R8	Jerarquía Envio y RegistroEnvios	calcularCosto; calcularTotal; equals; hashCode; toString	contratoDelSupertipoConservaCostoEstadoEHistorial; igualdadYHashPermanecenEstablesAlCambiarEstado; registroVacioYMixtoCalculanTotalMedianteSupertipo	Aprobado

Refactorizaciones y conservación del comportamiento

Primera refactorización

Problema: la fórmula base + peso × precio por kg estaba duplicada en ambos subtipos. Cambio: extraer el componente común a Tarifa.calcularComponente. El UML incorpora esa operación con visibilidad de paquete. Beneficio: una sola definición de la fórmula, conservando el redondeo final y el recargo específico de cada subtipo.

```
// Antes: expresión repetida en ambos subtipos.
getTarifa().getBase().add(getPesoKg().multiply(getTarifa().getPorKg()))

// Después: delegación a Tarifa.
getTarifa().calcularComponente(getPesoKg())

// Método extraído en Tarifa.
BigDecimal calcularComponente(BigDecimal pesoKg) {
    return base.add(pesoKg.multiply(porKg));
}
```

Pruebas protectoras: contratoDelSupertipoConservaCostoEstadoEHistorial, aceptaPesosValidosCercanosALosLimites y aceptaTarifaMinimaYRedondeaUnaSolaVez. Resultado antes y después: 31 casos aprobados. Evidencia: 01_linea_base.txt, 02_refactor_tarifa.txt y docs/refactor_1.diff.

Segunda refactorización

Problema: registrar reunía las comprobaciones de admisión y la escritura en el mapa. Cambio: extraer validarRegistro y mantener la inserción en registrar. El UML incorpora el nuevo método privado sin modificar asociaciones ni multiplicidades. Beneficio: hacer explícita la secuencia de validación e inserción y localizar las reglas de admisión.

```
// Antes
public void registrar(Envio envio) {
    if (envio == null) throw new IllegalArgumentException("Envío obligatorio");
    if (envios.containsKey(envio.getGuia())) {
        throw new IllegalArgumentException("Guía duplicada: " + envio.getGuia());
    }
    envios.put(envio.getGuia(), envio);
}
// Después
public void registrar(Envio envio) {
    validarRegistro(envio);
    envios.put(envio.getGuia(), envio);
}
private void validarRegistro(Envio envio) {
    if (envio == null) throw new IllegalArgumentException("Envío obligatorio");
    if (envios.containsKey(envio.getGuia())) {
        throw new IllegalArgumentException("Guía duplicada: " + envio.getGuia());
    }
}
```

Pruebas protectoras: registraEnviosDistintosYRechazaGuiaDuplicadaSinAlterarRegistro y rechazaRegistroNuloYConservaRegistroVacio. Resultado antes y después: 31 casos aprobados. Evidencia: 02_refactor_tarifa.txt, 03_refactor_registro.txt y docs/refactor_2.diff. Las pruebas y sus expectativas se conservaron en ambas etapas.

Discusión de diseño

La herencia se utiliza para representar dos formas de envío que comparten identidad, peso, ruta, tarifa y seguimiento. La decisión se sostiene en el contrato de comportamiento: ambos subtipos calculan un costo positivo en pesos colombianos sin modificar su estado y admiten la misma secuencia de entrega. El registro trabaja con referencias Envio, por lo que puede totalizar una colección mixta sin conocer las clases concretas. Las pruebas de contrato verifican esta sustitución con resultados monetarios específicos.

La composición se reserva para los eventos de seguimiento. Cada envío crea sus eventos, conserva su historial y no ofrece operaciones para incorporar objetos externos. Las rutas y las tarifas, en cambio, son asociaciones compartibles: sus instancias son inmutables y no dependen de un envío particular. Esta distinción evita atribuir propiedad fuerte a elementos que tienen existencia independiente. La visibilidad privada de los atributos y las copias no modificables impiden que un cliente elimine eventos o inserte envíos eludiendo las validaciones del registro.

La estrategia de pruebas prioriza reglas observables. Se comprobaron entradas válidas, nulos, duplicados, límites de peso, precisión monetaria y transiciones inválidas. Los nombres describen la condición o el comportamiento esperado. El cambio deliberado del flete demostró que la suite detecta al menos una alteración concreta de la fórmula; no permite afirmar que todos los defectos posibles estén cubiertos. Las refactorizaciones mantuvieron las pruebas y sus expectativas: primero se eliminó la fórmula duplicada y luego se separó la validación del registro. Los reportes antes y después respaldan la conservación del comportamiento.

La solución tiene limitaciones. El registro está en memoria y no es seguro para accesos concurrentes; dos registros independientes pueden contener una misma guía. El historial representa estados, pero no registra fechas ni responsables. Las rutas no verifican fronteras y los importes internacionales son supuestos académicos. Como mejora futura, se propone persistencia transaccional con unicidad global de guía y una política de recargos configurable. Estas extensiones deberían añadirse con nuevas reglas y pruebas, sin presentarlas como una refactorización que conserva exactamente el alcance actual.

Auditoría de coherencia

Los nombres de clases, operaciones y paquete coinciden entre UML y Java. Las asociaciones obligatorias se reflejan en constructores que rechazan null. La secuencia de entrega limita el historial a tres eventos. R1 a R8 tienen pruebas y ambas refactorizaciones están reflejadas en el diagrama final. La revisión limpia recompiló el proyecto y aprobó la suite completa.

Repositorio y referencias

Contenido del repositorio

El repositorio contiene pom.xml, README.md, src/main/java/co/edu/eseit/logistica, src/test/java/co/edu/eseit/logistica, docs y evidencias. El README registra el propósito, los integrantes, requisitos, estructura e instrucciones de ejecución. docs incluye el modelo UML editable, la matriz de trazabilidad y los cambios de refactorización; evidencias conserva los reportes de cada etapa.

Requisitos de ejecución: JDK 21, Maven 3.9.9 e internet para la primera descarga de dependencias. Los comandos se ejecutan desde la carpeta que contiene pom.xml:

```
mvn -B -ntp clean test
java -cp target/classes co.edu.eseit.logistica.Main
```

La salida de Main muestra COL-101 con 14000.00 COP, INT-901 con 64000.00 COP y un total de 78000.00 COP. Los nuevos resultados JUnit quedan en target/surefire-reports. El enlace del repositorio se registra en la portada.

Referencias

Escuela Superior de Empresa Ingeniería y Tecnología. Guía didáctica de la actividad 06: Diseño probado de una solución orientada a objetos Guía de curso.

Escuela Superior de Empresa Ingeniería y Tecnología. Modelado UML, pruebas unitarias y refactorización de jerarquías (TG-04, pp. 28–33) Material de curso.

JUnit Team. (2024). JUnit 5 user guide (Version 5.11.4). https://docs.junit.org/5.11.4/user-guide/

Object Management Group. (2017). Unified Modeling Language (Version 2.5.1). https://www.omg.org/spec/UML/2.5.1

Oracle. (2023). The Java language specification: Java SE 21 edition (Chapter 8, Classes). https://docs.oracle.com/javase/specs/jls/se21/html/jls-8.html

El vínculo entre reglas, modelo, código, pruebas y refactorizaciones sigue el TG-04 (ESEIT, s. f.-b, pp. 28–31). La notación UML se apoya en OMG (2017), la sobrescritura en Oracle (2023) y las pruebas parametrizadas en JUnit Team (2024).